

Skill-11

```
himasree@LAPTOP-41COT5AA:~/OSLab$ nano history_pipeline.c
himasree@LAPTOP-41COT5AA:~/OSLab$ gcc history_pipeline.c -o history_pipeline
./history_pipeline
=====
SHELL HISTORY + PIPELINE DEMO
=====

===== COMMAND HISTORY =====
1  pwd
2  ls
3  echo Hello
4  date
=====
History [2]: ls

===== PIPELINE =====
[1] ls | [2] grep .c | [3] wc -l
=====

Pipeline is valid.
Executing pipeline...

100

===== PIPELINE =====
[1] echo Hello World | [2] wc -w
=====

Second pipeline is valid.
Executing pipeline...
```

```
himasree@LAPTOP-41COT5AA:~/OSLab$ cat history_pipeline.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_HISTORY 10
#define MAX_COMMANDS 10
#define MAX_ARGS 50

/* =====
COMMAND HISTORY
===== */

char history[MAX_HISTORY][MAX_INPUT];
int history_count = 0;

/* Store command in history */
void add_history(const char *command)
{
    if (strlen(command) == 0)
        return;
```

```

/* Store command in history */
void add_history(const char *command)
{
    if (strlen(command) == 0)
        return;

    /*
     * If buffer is full, remove the oldest command
     * by shifting all entries one position.
     */
    if (history_count == MAX_HISTORY) {
        for (int i = 1; i < MAX_HISTORY; i++) {
            strcpy(history[i - 1], history[i]);
        }

        history_count--;
    }

    strcpy(history[history_count], command);
    history_count++;
}

/* Display history */
void show_history()
{
    printf("\n===== COMMAND HISTORY =====\n");
}

```

```

    for (int i = 0; i < history_count; i++) {
        printf("%d %s\n", i + 1, history[i]);
    }

    printf("=====\n");
}

/* Retrieve a history entry */
void get_history(int number)
{
    if (number < 1 || number > history_count) {
        printf("History entry does not exist.\n");
        return;
    }

    printf("History [%d]: %s\n",
        number,
        history[number - 1]);
}

/* =====
PIPELINE STRUCTURE
===== */

typedef struct {
    char command[MAX_INPUT];
} Command;

```

```

typedef struct {
    char command[MAX_INPUT];
} Command;

typedef struct {
    Command commands[MAX_COMMANDS];
    int command_count;
} Pipeline;

/* Create pipeline */
void init_pipeline(Pipeline *pipeline)
{
    pipeline->command_count = 0;
}

/* Add command to pipeline */
int add_pipeline_command(Pipeline *pipeline,
                        const char *command)
{
    if (pipeline->command_count >= MAX_COMMANDS) {

        printf("Pipeline capacity exceeded.\n");
        return 0;
    }

    strcpy(
        pipeline->commands[pipeline->command_count].command,

```

```

/* -----
   PIPELINE TEST 2
   ----- */

init_pipeline(&pipeline);

add_pipeline_command(
    &pipeline,
    "echo Hello World"
);

add_pipeline_command(
    &pipeline,
    "wc -w"
);

display_pipeline(&pipeline);

if (validate_pipeline(&pipeline)) {

    printf("\nSecond pipeline is valid.\n");
    printf("Executing pipeline...\n\n");

    execute_pipeline(&pipeline);
}

return 0;
}
himasree@LAPTOP-41COT5AA:~/OSLab$ |

```